

700106 / 700120 Lab Book

Vulkan Exercises

Exercise 1: PREPARING THE APPLICATION FOR TEXTURES

Solution&Reflection: Prepared the Vulkan application to support texture mapping. I started by downloading and integrating the stb_image.h library, which allows the program to load image files as textures. To enable the implementation, I defined STB_IMAGE_IMPLEMENTATION in one of the C++ source files before including the header. Next, I added a sample texture (a simple image file) to the project's build directory so the program could access it during runtime. The Vertex structure was updated to include a glm::vec2 texCoord attribute, allowing each vertex to store UV coordinates that map the texture correctly to the 3D model. I also modified the vertex input descriptions to account for this new attribute, ensuring Vulkan interprets the texture coordinates properly. Finally, I updated the cube's vertex data to include UV mappings for all six faces, assigning appropriate coordinates in the [0,1] range to make the texture appear correctly on each side.

This task helped me understand how texture coordinates relate to vertex data and how shaders use them to map 2D images onto 3D geometry. It also clarified how libraries like stb_image simplify texture loading.

Question: When defining UV coordinates for complex models, is there an efficient or automated way to generate them, or is manual assignment (like with the cube) still common practice in small projects?

```
64
65 // --- Vertex Data ---
66
67 struct Vertex {
68     glm::vec3 pos;
69     glm::vec3 color;
70     glm::vec3 normal;
71     glm::vec2 texCoord;
72
73     static VkVertexInputBindingDescription getBindingDescription() {
74         VkVertexInputBindingDescription bindingDescription{};
75         bindingDescription.binding = 0;
76         bindingDescription.stride = sizeof(Vertex);
77         bindingDescription.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;
78         return bindingDescription;
79     }
80
81     static std::array<VkVertexInputAttributeDescription, 4> getAttributeDescriptions() {
82         std::array<VkVertexInputAttributeDescription, 4> attributeDescriptions{};
83         attributeDescriptions[0] = { 0, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, pos) };
84         attributeDescriptions[1] = { 1, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, color) };
85         attributeDescriptions[2] = { 2, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, normal) };
86         attributeDescriptions[3] = { 3, 0, VK_FORMAT_R32G32_SFLOAT, offsetof(Vertex, texCoord) };
87         return attributeDescriptions;
88     }
89 };
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

```
25
26 #define STB_IMAGE_IMPLEMENTATION
27 #include <stb_image.h>
```

```

129
130  v const std::vector<Vertex> Cube_vertices = {
131      // Front face
132      {{-0.5f, -0.5f, 0.5f}, {1, 0, 0}, {0, 0, 1}, {0.0f, 0.0f}},
133      {{ 0.5f, -0.5f, 0.5f}, {0, 1, 0}, {0, 0, 1}, {1.0f, 0.0f}},
134      {{ 0.5f, 0.5f, 0.5f}, {0, 0, 1}, {0, 0, 1}, {1.0f, 1.0f}},
135      {{-0.5f, 0.5f, 0.5f}, {1, 1, 1}, {0, 0, 1}, {0.0f, 1.0f}},
136      {{-0.5f, -0.5f, 0.5f}, {1, 0, 0}, {0, 0, 1}, {0.0f, 0.0f}},
137
138      // Back face
139      {{-0.5f, -0.5f, -0.5f}, {1, 0, 0}, {0, 0, -1}, {1.0f, 0.0f}},
140      {{-0.5f, 0.5f, -0.5f}, {1, 1, 1}, {0, 0, -1}, {1.0f, 1.0f}},
141      {{ 0.5f, 0.5f, -0.5f}, {0, 0, 1}, {0, 0, -1}, {0.0f, 1.0f}},
142      {{ 0.5f, -0.5f, -0.5f}, {0, 1, 0}, {0, 0, -1}, {0.0f, 0.0f}},
143      {{-0.5f, -0.5f, -0.5f}, {1, 0, 0}, {0, 0, -1}, {1.0f, 0.0f}},
144
145      // Left face
146      {{-0.5f, 0.5f, 0.5f}, {1, 1, 1}, {-1, 0, 0}, {1.0f, 1.0f}},
147      {{-0.5f, 0.5f, -0.5f}, {0, 0, 1}, {-1, 0, 0}, {0.0f, 1.0f}},
148      {{-0.5f, -0.5f, -0.5f}, {0, 1, 0}, {-1, 0, 0}, {0.0f, 0.0f}},
149      {{-0.5f, -0.5f, 0.5f}, {1, 0, 0}, {-1, 0, 0}, {1.0f, 0.0f}},
150
151      // Right face
152      {{ 0.5f, 0.5f, 0.5f}, {1, 1, 1}, {1, 0, 0}, {0.0f, 1.0f}},
153      {{ 0.5f, -0.5f, 0.5f}, {1, 0, 0}, {1, 0, 0}, {0.0f, 0.0f}},
154      {{ 0.5f, -0.5f, -0.5f}, {0, 1, 0}, {1, 0, 0}, {1.0f, 0.0f}},
155      {{ 0.5f, 0.5f, -0.5f}, {0, 0, 1}, {1, 0, 0}, {1.0f, 1.0f}},
156
157      // Top face
158      {{-0.5f, 0.5f, -0.5f}, {1, 0, 0}, {0, 1, 0}, {0.0f, 0.0f}},
159      {{-0.5f, 0.5f, 0.5f}, {0, 1, 0}, {0, 1, 0}, {0.0f, 1.0f}},
160      {{ 0.5f, 0.5f, 0.5f}, {0, 0, 1}, {0, 1, 0}, {1.0f, 1.0f}},
161      {{ 0.5f, 0.5f, -0.5f}, {1, 1, 1}, {0, 1, 0}, {1.0f, 0.0f}},
162
163      // Bottom face
164      {{-0.5f, -0.5f, -0.5f}, {1, 0, 0}, {0, -1, 0}, {0.0f, 0.0f}},
165      {{ 0.5f, -0.5f, -0.5f}, {0, 1, 0}, {0, -1, 0}, {1.0f, 0.0f}},
166      {{ 0.5f, -0.5f, 0.5f}, {0, 0, 1}, {0, -1, 0}, {1.0f, 1.0f}},
167
168      // Bottom face
169      {{-0.5f, -0.5f, 0.5f}, {1, 0, 0}, {0, -1, 0}, {0.0f, 0.0f}},
170
171      // Bottom face
172      {{-0.5f, -0.5f, -0.5f}, {1, 0, 0}, {0, -1, 0}, {0.0f, 0.0f}},
173      {{ 0.5f, -0.5f, -0.5f}, {0, 1, 0}, {0, -1, 0}, {1.0f, 0.0f}},
174      {{ 0.5f, -0.5f, 0.5f}, {0, 0, 1}, {0, -1, 0}, {1.0f, 1.0f}},

```

Exercise 2: LOADING AND CREATING VULKAN IMAGE RESOURCES

Solution&Reflection: I implemented the process of loading an image file and preparing it as a usable texture within Vulkan. First, I declared the necessary Vulkan objects — the **VkImage**, **VkDeviceMemory**, **VkImageView**, and **VkSampler** — to store and reference the texture data. Using the **stb_image** library, I loaded the texture file into memory and calculated its size in bytes. Next, I created a **staging buffer**, which temporarily holds the image data on the CPU side. After mapping and copying the pixel data into this buffer, I freed the original CPU image memory to optimize resources. I then created a **VkImage** on the GPU with the appropriate format (**VK_FORMAT_R8G8B8A8_SRGB**) and allocated **device-local memory** to store it efficiently for rendering. The next step involved recording and submitting **command buffers** to handle layout transitions and copy operations — moving the texture from the staging buffer to the GPU image and changing its layout to be readable by shaders. Once the transfer was complete, I cleaned up the staging resources. Finally, I created an **image view** and **sampler**, which define how the texture will be accessed and filtered in the fragment shader, allowing for smooth, repeated texture mapping across surfaces.

Through this process, I gained a deeper understanding of how Vulkan handles image data and memory management. The staged copy and layout transition steps, while complex, demonstrated Vulkan's low-level control over GPU resources and synchronization. Setting up the sampler also clarified how filtering and anisotropy affect texture quality during rendering.

Question : Could you elaborate on how Vulkan's layout transitions differ from OpenGL's implicit state changes, and why explicit transitions are considered advantageous in modern rendering pipelines?

```

1520 void HelloTriangleApplication::createTextureFromFile(const std::string& filename)
1521 {
1522     int texWidth, texHeight, texChannels;
1523     stbi_uc* pixels = stbi_load(filename.c_str(), &texWidth, &texHeight, &texChannels, STBI_rgb_alpha);
1524     if (!pixels) {
1525         throw std::runtime_error(std::string("failed to load texture image: ") + filename);
1526     }
1527     VkDeviceSize imageSize = static_cast<VkDeviceSize>(texWidth) * static_cast<VkDeviceSize>(texHeight) * 4;
1528
1529     // staging buffer
1530     VkBuffer stagingBuffer;
1531     VkDeviceMemory stagingBufferMemory;
1532     createBuffer(imageSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
1533         VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
1534         stagingBuffer, stagingBufferMemory);
1535
1536     void* data;
1537     vkMapMemory(device, stagingBufferMemory, 0, imageSize, 0, &data);
1538     memcpy(data, pixels, static_cast<size_t>(imageSize));
1539     vkUnmapMemory(device, stagingBufferMemory);
1540     stbi_image_free(pixels);
1541
1542     // create image
1543     VkImage textureImageLocal;
1544     VkDeviceMemory textureImageMemoryLocal;
1545
1546     VkImageCreateInfo imageInfo{};
1547     imageInfo.sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO;
1548     imageInfo.imageType = VK_IMAGE_TYPE_2D;
1549     imageInfo.extent.width = static_cast<uint32_t>(texWidth);
1550     imageInfo.extent.height = static_cast<uint32_t>(texHeight);
1551     imageInfo.extent.depth = 1;
1552     imageInfo.mipLevels = 1;
1553     imageInfo.arrayLayers = 1;
1554     imageInfo.format = VK_FORMAT_R8G8B8A8_SRGB;
1555     imageInfo.tiling = VK_IMAGE_TILING_OPTIMAL;
1556     imageInfo.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
1557     imageInfo.usage = VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT;
1558     imageInfo.samples = VK_SAMPLE_COUNT_1_BIT;
1559     imageInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
1560
1561     if (vkCreateImage(device, &imageInfo, nullptr, &textureImageLocal) != VK_SUCCESS) {
1562         throw std::runtime_error("failed to create image!");
1563     }
1564
1565     VkMemoryRequirements memRequirements;
1566     vkGetImageMemoryRequirements(device, textureImageLocal, &memRequirements);
1567     VkMemoryAllocateInfo allocInfo{};
1568     allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
1569     allocInfo.allocationSize = memRequirements.size;
1570     allocInfo.memoryTypeIndex = findMemoryType(memRequirements.memoryTypeBits, VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT);
1571     if (vkAllocateMemory(device, &allocInfo, nullptr, &textureImageMemoryLocal) != VK_SUCCESS) {
1572         throw std::runtime_error("failed to allocate image memory!");
1573     }
1574     vkBindImageMemory(device, textureImageLocal, textureImageMemoryLocal, 0);
1575

```

```

1649 void HelloTriangleApplication::transitionImageLayout(VkImage image, VkImageLayout oldLayout, VkImageLayout newLayout, VkImageAspectFlags aspectMask)
1650 {
1651     VkCommandBuffer commandBuffer = beginSingleTimeCommands();
1652     VkImageMemoryBarrier barrier{};
1653     barrier.sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER;
1654     barrier.oldLayout = oldLayout;
1655     barrier.newLayout = newLayout;
1656     barrier.srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
1657     barrier.dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
1658     barrier.image = image;
1659     barrier.subresourceRange.aspectMask = aspectMask;
1660     barrier.subresourceRange.baseMipLevel = 0;
1661     barrier.subresourceRange.levelCount = 1;
1662     barrier.subresourceRange.baseArrayLayer = 0;
1663     barrier.subresourceRange.layerCount = 1;
1664     VkPipelineStageFlags sourceStage;
1665     VkPipelineStageFlags destinationStage;
1666     if (oldLayout == VK_IMAGE_LAYOUT_UNDEFINED && newLayout == VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL) {
1667         barrier.srcAccessMask = 0;
1668         barrier.dstAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;
1669         sourceStage = VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT;
1670         destinationStage = VK_PIPELINE_STAGE_TRANSFER_BIT;
1671     }
1672     else if (oldLayout == VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL && newLayout == VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL) {
1673         barrier.srcAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;
1674         barrier.dstAccessMask = VK_ACCESS_SHADER_READ_BIT;
1675         sourceStage = VK_PIPELINE_STAGE_TRANSFER_BIT;
1676         destinationStage = VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT;
1677     }
1678     else if (oldLayout == VK_IMAGE_LAYOUT_UNDEFINED && newLayout == VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL) {
1679         barrier.srcAccessMask = 0;
1680         barrier.dstAccessMask = VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_READ_BIT | VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT;
1681         sourceStage = VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT;
1682         destinationStage = VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT;
1683     }
1684     else {
1685         throw std::invalid_argument("unsupported layout transition!");
1686     }
1687     vkCmdPipelineBarrier(commandBuffer, sourceStage, destinationStage, 0, 0, nullptr, 0, nullptr, 1, &barrier);
1688

```

```

1691
1692 void HelloTriangleApplication::copyBufferToImage(VkBuffer buffer, VkImage image, uint32_t width, uint32_t height) {
1693     VkCommandBuffer commandBuffer = beginSingleTimeCommands();
1694     VkBufferImageCopy region{};
1695     region.bufferOffset = 0;
1696     region.bufferRowLength = 0;
1697     region.bufferImageHeight = 0;
1698     region.imageSubresource.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
1699     region.imageSubresource.mipLevel = 0;
1700     region.imageSubresource.baseArrayLayer = 0;
1701     region.imageSubresource.layerCount = 1;
1702     region.imageOffset = { 0, 0, 0 };
1703     region.imageExtent = { width, height, 1 };
1704     vkCmdCopyBufferToImage(commandBuffer, buffer, image, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, 1, &region);
1705     endSingleTimeCommands(commandBuffer);
1706 }
1707
1708 void VkCommandBuffer HelloTriangleApplication::beginSingleTimeCommands() {
1709     VkCommandBufferAllocateInfo allocInfo{};
1710     allocInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO;
1711     allocInfo.level = VK_COMMAND_BUFFER_LEVEL_PRIMARY;
1712     allocInfo.commandPool = commandPool;
1713     allocInfo.commandBufferCount = 1;
1714     VkCommandBuffer commandBuffer;
1715     vkAllocateCommandBuffers(device, &allocInfo, &commandBuffer);
1716     VkCommandBufferBeginInfo beginInfo{};
1717     beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;
1718     beginInfo.flags = VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT;
1719     vkBeginCommandBuffer(commandBuffer, &beginInfo);
1720     return commandBuffer;
1721 }
1722
1723 void HelloTriangleApplication::endSingleTimeCommands(VkCommandBuffer commandBuffer) {
1724     vkEndCommandBuffer(commandBuffer);
1725     VkSubmitInfo submitInfo{};
1726     submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;
1727     submitInfo.commandBufferCount = 1;
1728     submitInfo.pCommandBuffers = &commandBuffer;
1729     vkQueueSubmit(graphicsQueue, 1, &submitInfo, VK_NULL_HANDLE);
1730     vkQueueWaitIdle(graphicsQueue);
1731     vkFreeCommandBuffers(device, commandPool, 1, &commandBuffer);
1732 }
1733
1734 // --- Callback Implementations ---
1735

```

Exercise 3: BINDING AND SHADER UPDATES

Question: Improve visual quality by moving the lighting logic to the fragment shader, allowing for smoother, more accurate calculations.

Solution: Integrated texture sampling into the Vulkan rendering pipeline by updating descriptor bindings and modifying the shaders. First, I extended the **descriptor set layout** to include a new binding for a combined image sampler, allowing the fragment shader to access texture data. I then updated the **descriptor pool** to allocate space for this new descriptor type and modified the descriptor set creation process to include both the **uniform buffer (binding = 0)** and the **texture sampler (binding = 1)**. This required setting up a `VkDescriptorImageInfo` structure that linked the texture image view and sampler created in the previous exercises. After ensuring the descriptor set layout included both bindings, I updated the cleanup routines to properly release these texture resources, avoiding potential memory leaks. I also added the texture setup calls to `initVulkan()` so that all texture resources were prepared before rendering.

Next, I modified the **vertex** and **fragment shaders** to support texture coordinates and sampling. In the vertex shader, I introduced a new input for the texture coordinates and passed it to the fragment shader. The fragment shader was then updated to include a **sampler2D** uniform and to sample the texture using the interpolated coordinates. The final color output combined the sampled texture color with existing lighting calculations to create a more realistic, lit, textured cube.

Question: When working with multiple textures or material types in a single scene, would it be more efficient to use a single descriptor set with multiple bindings, or to create separate descriptor sets per material or object?

```
674
675 void HelloTriangleApplication::createDescriptorSetLayout() {
676     VkDescriptorSetLayoutBinding uboLayoutBinding{};
677     uboLayoutBinding.binding = 0;
678     uboLayoutBinding.descriptorCount = 1;
679     uboLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER; // <- fixed
680     uboLayoutBinding.stageFlags = VK_SHADER_STAGE_VERTEX_BIT;
681     uboLayoutBinding.pImmutableSamplers = nullptr;
682
683     // Sampler binding at 1
684     VkDescriptorSetLayoutBinding samplerLayoutBinding{};
685     samplerLayoutBinding.binding = 1;
686     samplerLayoutBinding.descriptorCount = 1;
687     samplerLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
688     samplerLayoutBinding.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;
689     samplerLayoutBinding.pImmutableSamplers = nullptr;
690
691     std::array<VkDescriptorSetLayoutBinding, 2> bindings = { uboLayoutBinding, samplerLayoutBinding };
692
693     VkDescriptorSetLayoutCreateInfo layoutInfo{};
694     layoutInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO;
695     layoutInfo.bindingCount = static_cast<uint32_t>(bindings.size());
696     layoutInfo.pBindings = bindings.data();
697
698     if (vkCreateDescriptorSetLayout(device, &layoutInfo, nullptr, &descriptorSetLayout) != VK_SUCCESS) {
699         throw std::runtime_error("Failed to create descriptor set layout!");
700     }
701 }
```



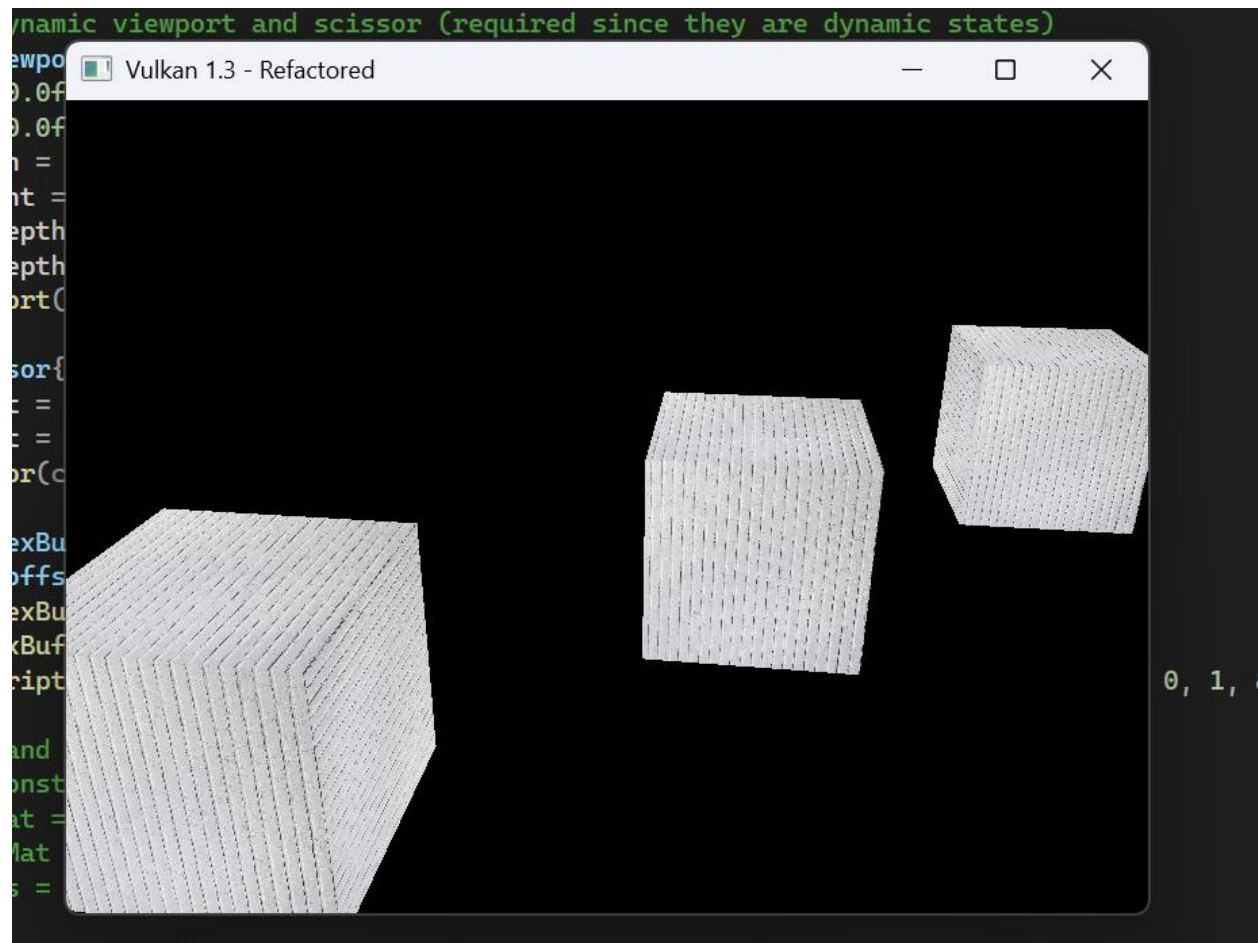
```

901
902 void HelloTriangleApplication::createDescriptorSets() {
903     std::vector<VkDescriptorSetLayout> layouts(MAX_FRAMES_IN_FLIGHT, descriptorSetLayout);
904     VkDescriptorSetAllocateInfo allocInfo{};
905     allocInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_ALLOCATE_INFO;
906     allocInfo.descriptorPool = descriptorPool;
907     allocInfo.descriptorSetCount = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);
908     allocInfo.pSetLayouts = layouts.data();
909
910     descriptorSets.resize(MAX_FRAMES_IN_FLIGHT);
911     if (vkAllocateDescriptorSets(device, &allocInfo, descriptorSets.data()) != VK_SUCCESS) {
912         throw std::runtime_error("Failed to allocate descriptor sets!");
913     }
914
915     VkDescriptorImageInfo imageInfo{};
916     imageInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
917     imageInfo.imageView = textureImageView;
918     imageInfo.sampler = textureSampler;
919
920     for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
921         VkDescriptorBufferInfo bufferInfo{};
922         bufferInfo.buffer = uniformBuffers[i];
923         bufferInfo.offset = 0;
924         bufferInfo.range = sizeof(SceneUniformBufferObject);
925
926         std::array<VkWriteDescriptorSet, 2> descriptorWrites{};
927
928         descriptorWrites[0].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
929         descriptorWrites[0].dstSet = descriptorSets[i];
930         descriptorWrites[0].dstBinding = 0;
931         descriptorWrites[0].dstArrayElement = 0;
932         descriptorWrites[0].descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
933         descriptorWrites[0].descriptorCount = 1;
934         descriptorWrites[0].pBufferInfo = &bufferInfo;
935
936         descriptorWrites[1].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
937         descriptorWrites[1].dstSet = descriptorSets[i];
938         descriptorWrites[1].dstBinding = 1;
939         descriptorWrites[1].dstArrayElement = 0;
940         descriptorWrites[1].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
941         descriptorWrites[1].descriptorCount = 1;
942         descriptorWrites[1].pImageInfo = &imageInfo;
943
944         vkUpdateDescriptorSets(device, static_cast<uint32_t>(descriptorWrites.size()), descriptorWrites.data(), 0, nullptr);
945     }
946

```

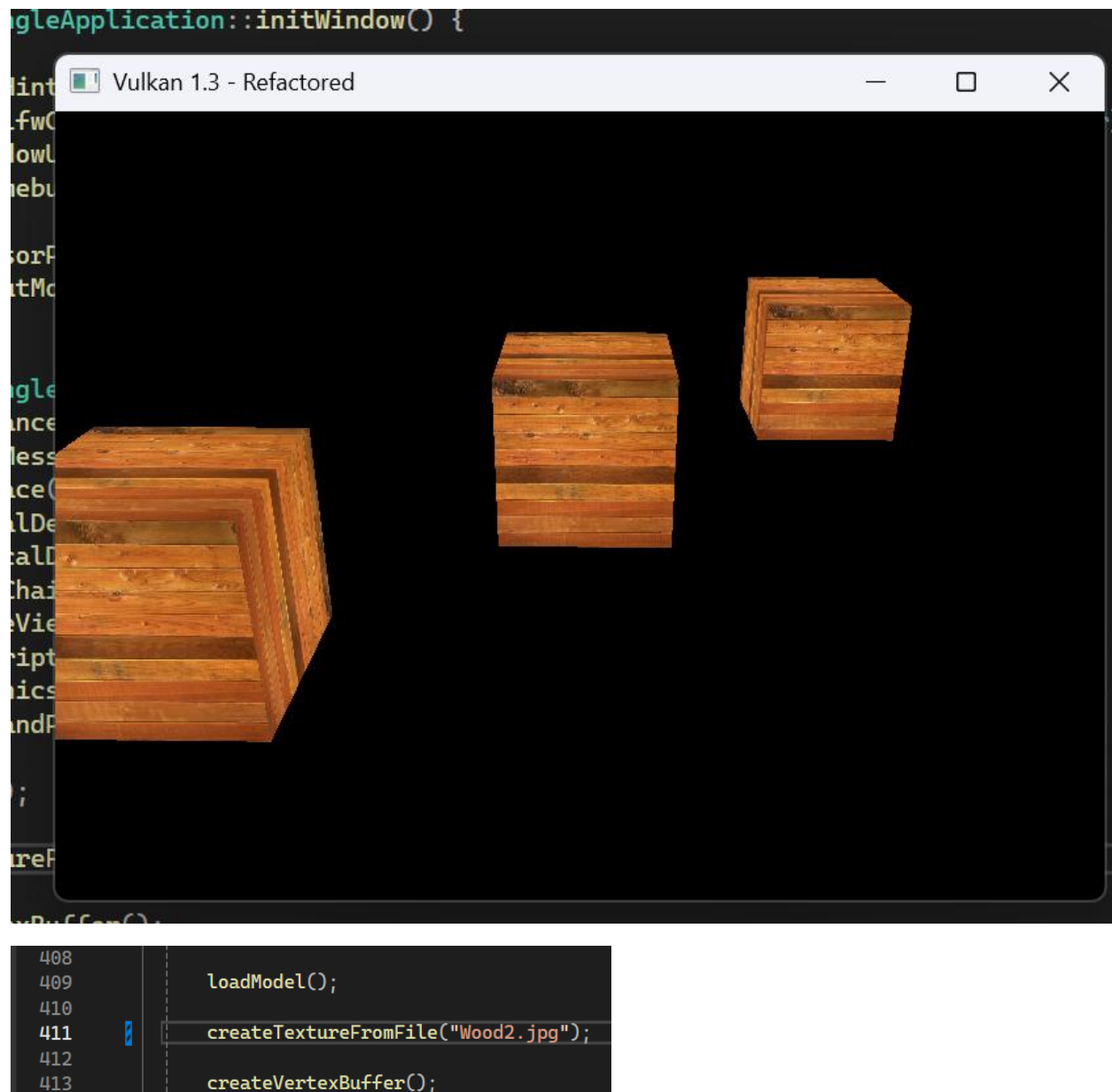
shader.frag	shader.vert	Lab_Tutorial_Template.cpp
1	#version 450	
2		
3	layout(location = 0) in vec3 inPosition;	
4	layout(location = 1) in vec3 inColor;	
5	layout(location = 2) in vec3 inNormal;	
6	layout(location = 3) in vec2 inTexCoord;	
7		
8	layout(set = 0, binding = 0) uniform SceneUBO {	
9	mat4 view;	
10	mat4 proj;	
11	vec3 lightPos1; // padding handled on CPU	
12	vec3 lightPos2;	
13	vec3 eyePos;	
14	} ubo;	
15		
16	layout(push_constant) uniform PushConstants {	
17	mat4 model;	
18	vec4 ambientMat;	
19	vec4 specularMat;	
20	float shininess;	
21	} pc;	
22		
23	layout(location = 0) out vec3 vWorldPos;	
24	layout(location = 1) out vec3 vNormal;	
25	layout(location = 3) out vec2 fragTexCoord;	
26		
27	void main() {	
28	mat4 model = pc.model;	
29	vec4 worldPos = model * vec4(inPosition, 1.0);	
30	vWorldPos = worldPos.xyz;	
31	vNormal = mat3(transpose(inverse(model))) * inNormal;	
32	fragTexCoord = inTexCoord;	
33	gl_Position = ubo.proj * ubo.view * worldPos;	
34	}	

```
1  #version 450
2
3  layout(location = 0) in vec3 vWorldPos;
4  layout(location = 1) in vec3 vNormal;
5  layout(location = 3) in vec2 fragTexCoord;
6
7  layout(location = 0) out vec4 outColor;
8
9  layout(set = 0, binding = 0) uniform SceneUBO {
10     mat4 view;
11     mat4 proj;
12     vec3 lightPos1; float _pad0;
13     vec3 lightPos2; float _pad1;
14     vec3 eyePos;    float _pad2;
15 } UBO;
16
17 layout(push_constant) uniform ObjectPush {
18     mat4 model;
19     vec4 ambientMat;
20     vec4 specularMat;
21     float shininess;
22     float _pcPad0; float _pcPad1; float _pcPad2;
23 } PC;
24
25 layout(binding = 1) uniform sampler2D texSampler;
26
27 void main() {
28     vec3 N = normalize(vNormal);
29     vec3 V = normalize(UBO.eyePos - vWorldPos);
30
31     // Base albedo: white (ignores per-vertex color)
32     vec3 baseDiffuse = vec3(1.0);
33
34     // Light colors
35     vec3 light1Color = vec3(1.0, 1.0, 1.0); // white
36     vec3 light2Color = vec3(1.0, 0.1, 0.1); // red
37
38     // Light 1
39     vec3 L1 = normalize(UBO.lightPos1 - vWorldPos);
40     float diff1 = max(dot(N, L1), 0.0);
41     vec3 H1 = normalize(L1 + V);
42     float spec1 = pow(max(dot(N, H1), 0.0), PC.shininess);
43
44     // Light 2
45     vec3 L2 = normalize(UBO.lightPos2 - vWorldPos);
46     float diff2 = max(dot(N, L2), 0.0);
47     vec3 H2 = normalize(L2 + V);
48     float spec2 = pow(max(dot(N, H2), 0.0), PC.shininess);
49
50     vec4 texColor = texture(texSampler, fragTexCoord);
51
52     vec3 ambient = PC.ambientMat.rgb;
53     vec3 diffuse = baseDiffuse * (diff1 * light1Color + diff2 * light2Color);
54     vec3 specular = PC.specularMat.rgb * (spec1 * light1Color + spec2 * light2Color);
55
56     vec3 color = (ambient + diffuse) * texColor.rgb + specular;
57     outColor = vec4(color, 1.0);
58 }
```

Exercise 4: A WOODEN CUBE

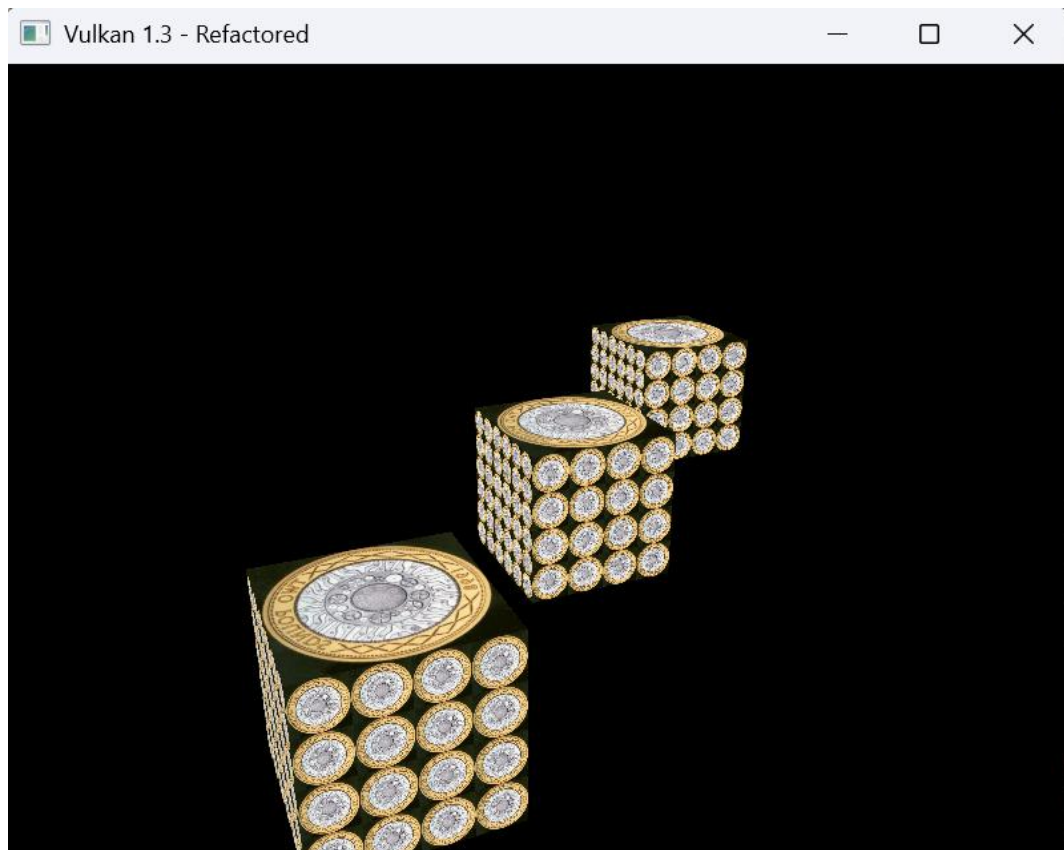
Solution: Replaced the existing texture in the Vulkan application with a new wood texture to create a more realistic, wooden appearance for the cube. The process began by converting the provided wood.dds texture file into a more commonly supported JPEG format (wood.jpg) using an online texture converter. Once converted, I replaced the previous texture.jpg file in the project with the new wood texture and ensured that the application loaded it correctly. In the rendering pipeline, I modified the fragment shader from the previous per-fragment lighting implementation so that the cube's surface color now comes directly from the wood texture instead of the vertex color attribute. This adjustment allowed the texture to interact with the lighting model, producing visually realistic shading, highlights, and wood grain detail across the cube's surfaces.



Exercise 5: TEXTURE WRAPPING MODE

Solution & Reflection: I implemented a textured cube using the **Coin.jpg** texture, experimenting with different **texture wrapping modes** and UV scaling to display varying numbers of coin patterns on each cube face. To achieve this, I introduced a small lambda function `auto faceUV = [&](float S, glm::vec2 uv) { return uv * S; };` that scales the texture coordinates (uv) by a specified factor S. This allowed each face of the cube to have independent control over how many times the texture repeats. For example, I used `faceUV(1.f, uv)` on the front face for a single coin pattern, while the back face used `faceUV(2.f, uv)` to repeat the texture twice. Because the texture sampler was configured with `VK_SAMPLER_ADDRESS_MODE_REPEAT`, any UV coordinates outside the `[0,1]` range caused the texture to wrap and tile seamlessly. The result was a cube where each side displayed a different number of repeated coin textures, clearly demonstrating how UV scaling and texture wrapping interact in Vulkan.

Question: If we wanted to apply different wrapping modes (e.g., `CLAMP_TO_EDGE` or `MIRRORED_REPEAT`) to individual faces of a model, would that require multiple samplers and descriptor bindings, or can it be achieved using a single texture and shader logic?



```

auto faceUV = [&](float S, glm::vec2 uv) { return uv * S; };
const std::vector<Vertex> Cube_vertices = {
    // Front face
    {{-0.5f, -0.5f, 0.5f}, {1, 0, 0}, {0, 0, 1}, faceUV(1.f, {0.0f, 0.0f})},
    {{ 0.5f, -0.5f, 0.5f}, {0, 1, 0}, {0, 0, 1}, faceUV(1.f, {1.0f, 0.0f})},
    {{ 0.5f, 0.5f, 0.5f}, {0, 0, 1}, {0, 0, 1}, faceUV(1.f, {1.0f, 1.0f})},
    {{-0.5f, 0.5f, 0.5f}, {0, 0, 1}, {0, 0, 1}, faceUV(1.f, {0.0f, 1.0f})},
    {{-0.5f, -0.5f, 0.5f}, {1, 1, 1}, {0, 0, 1}, faceUV(1.f, {0.0f, 0.0f})},
    {{-0.5f, -0.5f, 0.5f}, {1, 0, 0}, {0, 0, 1}, faceUV(1.f, {0.0f, 0.0f})},

    // Back face
    {{-0.5f, -0.5f, -0.5f}, {1, 0, 0}, {0, 0, -1}, faceUV(2.f, {1.0f, 0.0f})},
    {{-0.5f, 0.5f, -0.5f}, {1, 1, 1}, {0, 0, -1}, faceUV(2.f, {1.0f, 1.0f})},
    {{ 0.5f, 0.5f, -0.5f}, {0, 0, 1}, {0, 0, -1}, faceUV(2.f, {0.0f, 1.0f})},
    {{ 0.5f, 0.5f, -0.5f}, {0, 0, 1}, {0, 0, -1}, faceUV(2.f, {0.0f, 1.0f})},
    {{ 0.5f, -0.5f, -0.5f}, {0, 1, 0}, {0, 0, -1}, faceUV(2.f, {0.0f, 0.0f})},
    {{-0.5f, -0.5f, -0.5f}, {1, 0, 0}, {0, 0, -1}, faceUV(2.f, {1.0f, 0.0f})},

    // Left face
    {{-0.5f, 0.5f, 0.5f}, {1, 1, 1}, {-1, 0, 0}, faceUV(3.f, {1.0f, 1.0f})},
    {{-0.5f, 0.5f, -0.5f}, {0, 0, 1}, {-1, 0, 0}, faceUV(3.f, {0.0f, 1.0f})},
    {{-0.5f, -0.5f, -0.5f}, {0, 1, 0}, {-1, 0, 0}, faceUV(3.f, {0.0f, 0.0f})},
    {{-0.5f, -0.5f, 0.5f}, {0, 1, 0}, {-1, 0, 0}, faceUV(3.f, {0.0f, 0.0f})},
    {{-0.5f, -0.5f, 0.5f}, {1, 0, 0}, {-1, 0, 0}, faceUV(3.f, {1.0f, 0.0f})},
    {{-0.5f, 0.5f, 0.5f}, {1, 1, 1}, {-1, 0, 0}, faceUV(3.f, {1.0f, 1.0f})},

    // Right face
    {{ 0.5f, 0.5f, 0.5f}, {1, 1, 1}, {1, 0, 0}, faceUV(4.f, {0.0f, 1.0f})},
    {{ 0.5f, -0.5f, 0.5f}, {1, 0, 0}, {1, 0, 0}, faceUV(4.f, {0.0f, 0.0f})},
    {{ 0.5f, -0.5f, -0.5f}, {0, 1, 0}, {1, 0, 0}, faceUV(4.f, {1.0f, 0.0f})},
    {{ 0.5f, -0.5f, -0.5f}, {0, 1, 0}, {1, 0, 0}, faceUV(4.f, {1.0f, 0.0f})},
    {{ 0.5f, 0.5f, -0.5f}, {0, 0, 1}, {1, 0, 0}, faceUV(4.f, {1.0f, 1.0f})},
    {{ 0.5f, 0.5f, 0.5f}, {1, 1, 1}, {1, 0, 0}, faceUV(4.f, {0.0f, 1.0f})},

    // Top face
    {{-0.5f, 0.5f, -0.5f}, {1, 0, 0}, {0, 1, 0}, faceUV(5.f, {0.0f, 0.0f})},
    {{-0.5f, 0.5f, 0.5f}, {0, 1, 0}, {0, 1, 0}, faceUV(5.f, {0.0f, 1.0f})},
    {{ 0.5f, 0.5f, 0.5f}, {0, 0, 1}, {0, 1, 0}, faceUV(5.f, {1.0f, 1.0f})},
    {{ 0.5f, 0.5f, 0.5f}, {0, 0, 1}, {0, 1, 0}, faceUV(5.f, {1.0f, 1.0f})},
    {{ 0.5f, 0.5f, -0.5f}, {1, 1, 1}, {0, 1, 0}, faceUV(5.f, {1.0f, 0.0f})},
    {{-0.5f, 0.5f, -0.5f}, {1, 0, 0}, {0, 1, 0}, faceUV(5.f, {0.0f, 0.0f})},

    // Bottom face
    {{-0.5f, -0.5f, -0.5f}, {1, 0, 0}, {0, -1, 0}, faceUV(6.f, {0.0f, 0.0f})},
    {{ 0.5f, -0.5f, -0.5f}, {0, 1, 0}, {0, -1, 0}, faceUV(6.f, {1.0f, 0.0f})},
    {{ 0.5f, -0.5f, 0.5f}, {0, 0, 1}, {0, -1, 0}, faceUV(6.f, {1.0f, 1.0f})},
    {{ 0.5f, -0.5f, 0.5f}, {0, 0, 1}, {0, -1, 0}, faceUV(6.f, {1.0f, 1.0f})},
    {{-0.5f, -0.5f, 0.5f}, {1, 1, 1}, {0, -1, 0}, faceUV(6.f, {0.0f, 1.0f})},
    {{-0.5f, -0.5f, -0.5f}, {1, 0, 0}, {0, -1, 0}, faceUV(6.f, {0.0f, 0.0f})}
};

```

Exercise 6: TEXTURE FILTERING TECHNIQUES

Solution: I implemented real-time filter switching within the `processInput()` function. By pressing keys 1–4, the user can toggle between *nearest-neighbour*, *bilinear*, *anisotropic*, and *bicubic* filtering modes. This was achieved by defining a `textureFilterMode` variable that updates based on input, allowing the rendering pipeline to apply the corresponding sampler configuration. This interactivity made it easy to visually compare how each filtering mode handles minification and magnification.

Through observation, *nearest-neighbour filtering* produced a pixelated look, especially at shallow viewing angles. *Bilinear filtering* offered smoother results but introduced slight blurriness under large-scale minification. *Bicubic filtering* improved smoothness further, providing more natural gradients. However, the most visually effective was *anisotropic filtering*, which preserved detail along the elongated road and reduced stretching artifacts, demonstrating why it is widely used in 3D graphics for textured surfaces viewed at oblique angles.

This exercise reinforced the critical role of texture filtering in maintaining visual fidelity and realism in real-time rendering. It also showed how user-controllable filtering settings can be valuable for testing and optimizing graphical quality versus performance trade-offs.

Question: When implementing dynamic filter switching in more complex engines, is it common practice to re-create samplers at runtime, or are multiple preconfigured samplers typically bound and switched through descriptor updates for performance reasons?

```
92
93 struct SceneUniformBufferObject {
94     alignas(16) glm::mat4 view;
95     alignas(16) glm::mat4 proj;
96     alignas(16) glm::vec3 lightPos1; // Static White
97     alignas(16) glm::vec3 lightPos2; // Rotating Red
98     alignas(16) glm::vec3 eyePos;
99     // added for texture sampling control
100    alignas(8) glm::vec2 texSize;
101    alignas(4) int filterMode;
102};
```

```

120
121 // Texture filter modes
122 enum TextureFilterMode {
123     FILTER_NEAREST = 0,
124     FILTER_LINEAR = 1,
125     FILTER_ANISOTROPIC = 2,
126     FILTER_BICUBIC = 3
127 };
128

```

```

385 // Implementation
386
387 void HelloTriangleApplication::processInput(GLFWwindow* window) {
388     const float cameraSpeed = 2.5f * deltaTime;
389
390     // Build horizontal basis for Z-up
391     glm::vec3 right = glm::normalize(glm::cross(cameraFront, cameraUp));
392     glm::vec3 forward = glm::normalize(glm::cross(cameraUp, right));
393
394     if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS) cameraPos += cameraSpeed * forward;
395     if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS) cameraPos -= cameraSpeed * forward;
396     if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS) cameraPos -= cameraSpeed * right;
397     if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS) cameraPos += cameraSpeed * right;
398     if (glfwGetKey(window, GLFW_KEY_E) == GLFW_PRESS) cameraPos += cameraSpeed * cameraUp; // up
399     if (glfwGetKey(window, GLFW_KEY_Q) == GLFW_PRESS) cameraPos -= cameraSpeed * cameraUp; // down
400
401     // Filter mode switching (1=Nearest, 2=Linear, 3=Aniso, 4=Bicubic)
402     if (glfwGetKey(window, GLFW_KEY_1) == GLFW_PRESS) textureFilterMode = FILTER_NEAREST;
403     if (glfwGetKey(window, GLFW_KEY_2) == GLFW_PRESS) textureFilterMode = FILTER_LINEAR;
404     if (glfwGetKey(window, GLFW_KEY_3) == GLFW_PRESS) textureFilterMode = FILTER_ANISOTROPIC;
405     if (glfwGetKey(window, GLFW_KEY_4) == GLFW_PRESS) textureFilterMode = FILTER_BICUBIC;
406 }
407
408 void HelloTriangleApplication::run() {

```

```

1658 // Create texture sampler(sampler);
1659
1660 // save texture size for shader sampling logic
1661 loadedTexWidth = texWidth;
1662 loadedTexHeight = texHeight;
1663
1664 textureImage = textureImageLocal;
1665 textureImageMemory = textureImageMemoryLocal;
1666 textureImageView = createImageView(textureImageLocal, VK_FORMAT_R8G8B8A8_SRGB, VK_IMAGE_ASPECT_COLOR_BIT);
1667
1668 VkSampler localSampler;
1669 createTextureSampler(localSampler, textureFilterMode);
1670 textureSampler = localSampler;
1671

```

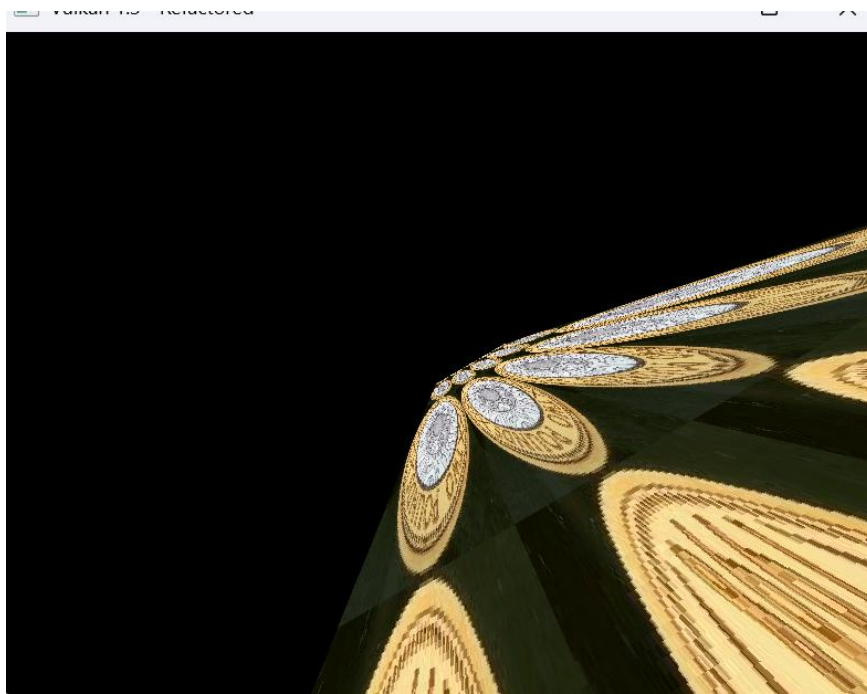


```

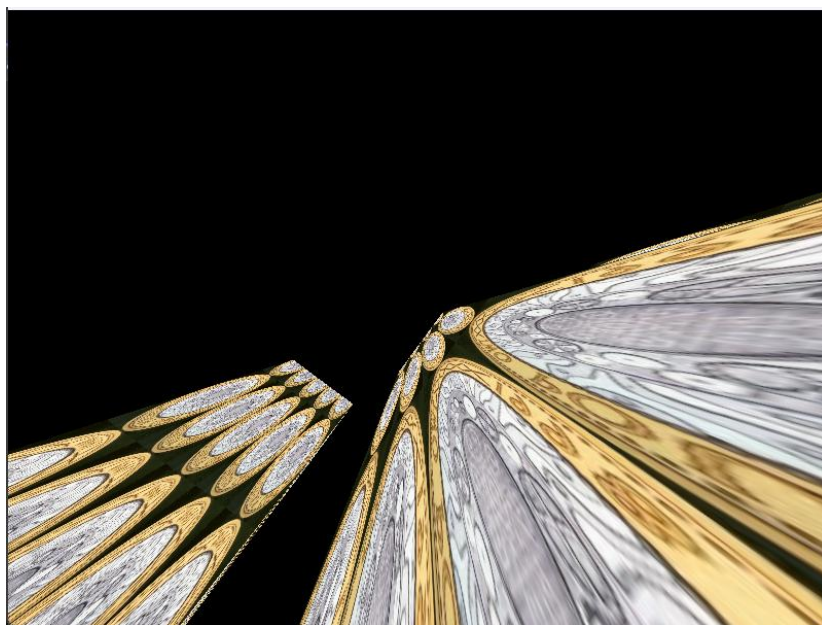
1677
1678 void HelloTriangleApplication::createTextureSampler(VkSampler& outSampler, int mode)
1679 {
1680     VkPhysicalDeviceProperties properties{};
1681     vkGetPhysicalDeviceProperties(physicalDevice, &properties);
1682
1683     VkSamplerCreateInfo samplerInfo{};
1684     samplerInfo.sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO;
1685     samplerInfo.magFilter = VK_FILTER_LINEAR;
1686     samplerInfo.minFilter = VK_FILTER_LINEAR;
1687     samplerInfo.addressModeU = VK_SAMPLER_ADDRESS_MODE_REPEAT;
1688     samplerInfo.addressModeV = VK_SAMPLER_ADDRESS_MODE_REPEAT;
1689     samplerInfo.addressModeW = VK_SAMPLER_ADDRESS_MODE_REPEAT;
1690     samplerInfo.anisotropyEnable = VK_TRUE;
1691     samplerInfo.maxAnisotropy = properties.limits.maxSamplerAnisotropy;
1692     samplerInfo.borderColor = VK_BORDER_COLOR_INT_OPAQUE_BLACK;
1693     samplerInfo.unnormalizedCoordinates = VK_FALSE;
1694     samplerInfo.compareEnable = VK_FALSE;
1695     samplerInfo.compareOp = VK_COMPARE_OP_ALWAYS;
1696     samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR;
1697     samplerInfo.mipLodBias = 0.0f;
1698     samplerInfo.minLod = 0.0f;
1699     samplerInfo.maxLod = 0.0f;
1700
1701     switch (mode) {
1702     case FILTER_NEAREST:
1703         samplerInfo.magFilter = VK_FILTER_NEAREST;
1704         samplerInfo.minFilter = VK_FILTER_NEAREST;
1705         samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_NEAREST;
1706         samplerInfo.anisotropyEnable = VK_FALSE;
1707         samplerInfo.maxAnisotropy = 1.0f;
1708         break;
1709     case FILTER_LINEAR:
1710         samplerInfo.magFilter = VK_FILTER_LINEAR;
1711         samplerInfo.minFilter = VK_FILTER_LINEAR;
1712         samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR;
1713         samplerInfo.anisotropyEnable = VK_FALSE;
1714         samplerInfo.maxAnisotropy = 1.0f;
1715         break;
1716     case FILTER_ANISOTROPIC:
1717         samplerInfo.magFilter = VK_FILTER_LINEAR;
1718         samplerInfo.minFilter = VK_FILTER_LINEAR;
1719         samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR;
1720         samplerInfo.anisotropyEnable = VK_TRUE;
1721         samplerInfo.maxAnisotropy = properties.limits.maxSamplerAnisotropy;
1722         break;
1723     case FILTER_BICUBIC:
1724         // bicubic is done manually in shader using texelFetch; sampler can be nearest
1725         samplerInfo.magFilter = VK_FILTER_NEAREST;
1726         samplerInfo.minFilter = VK_FILTER_NEAREST;
1727         samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_NEAREST;
1728         samplerInfo.anisotropyEnable = VK_FALSE;
1729         samplerInfo.maxAnisotropy = 1.0f;
1730         break;
1731     default:
1732         samplerInfo.magFilter = VK_FILTER_LINEAR;
1733         samplerInfo.minFilter = VK_FILTER_LINEAR;
1734         samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR;
1735         samplerInfo.anisotropyEnable = VK_FALSE;
1736         samplerInfo.maxAnisotropy = 1.0f;

```

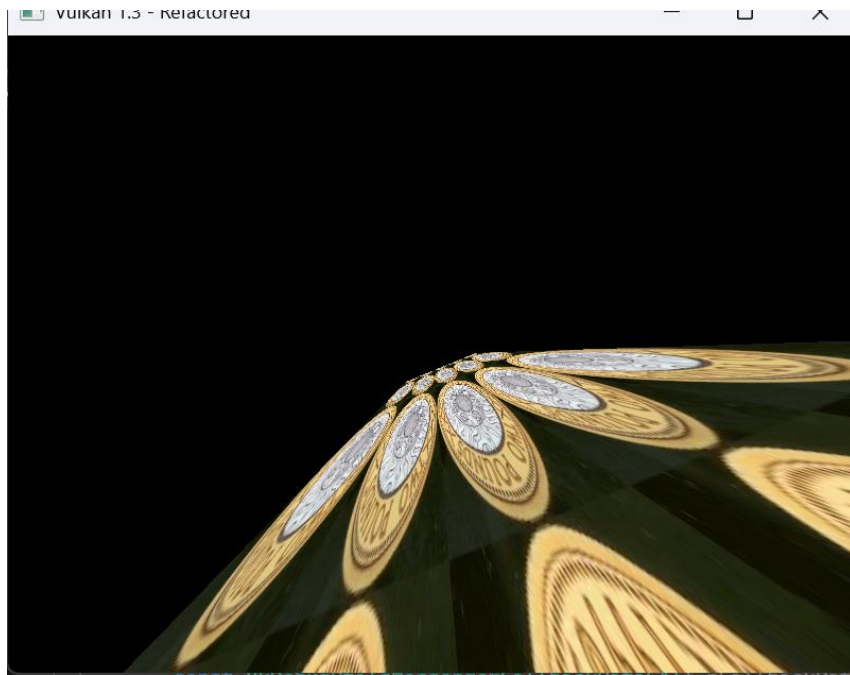

FILTER_NEAREST:



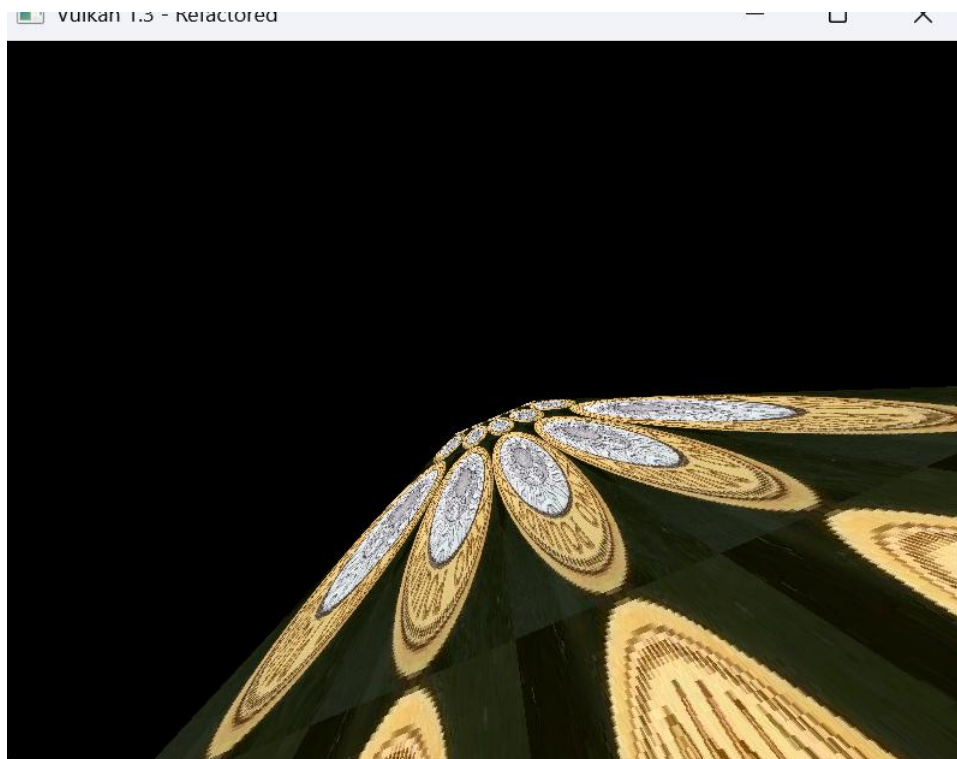
FILTER_LINEAR:



FILTER_ANISOTROPIC:



FILTER_BICUBIC:



EXERCISE 7. MULTIPLE TEXTURING

Solution & Reflection: This exercise demonstrated how multiple texture sampling can enhance surface realism by combining patterns or overlays.

It also shows how easily shaders can be extended to use multiple samplers by adding descriptor bindings.

Blending approaches (mix, multiply, add) each yield different visual results depending on desired style (e.g., roughness maps, decals, albedo overlays).

Question: How does Vulkan handle performance when sampling multiple textures per fragment — is there a limit or best practice?

```
void HelloTriangleApplication::updateDescriptorSetsSampler()
{
    if (descriptorSets.empty()) return;

    for (size_t i = 0; i < descriptorSets.size(); ++i) {
        VkDescriptorImageInfo coinInfo{};
        coinInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
        coinInfo.imageView = textureImageView;
        coinInfo.sampler = textureSampler;

        VkDescriptorImageInfo tileInfo{};
        tileInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
        tileInfo.imageView = tileTextureImageView;
        tileInfo.sampler = tileTextureSampler;

        std::array<VkWriteDescriptorSet, 2> writes{};

        // Rebind coin sampler
        writes[0].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
        writes[0].dstSet = descriptorSets[i];
        writes[0].dstBinding = 1;
        writes[0].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
        writes[0].descriptorCount = 1;
        writes[0].pImageInfo = &coinInfo;

        // Rebind tile sampler
        writes[1].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
        writes[1].dstSet = descriptorSets[i];
        writes[1].dstBinding = 2;
        writes[1].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
        writes[1].descriptorCount = 1;
        writes[1].pImageInfo = &tileInfo;

        vkUpdateDescriptorSets(device, static_cast<uint32_t>(writes.size()), writes.data(), 0, nullptr);
    }
}
```

```

980         coinInfo.sampler = coinTextureSampler;
981
982         std::array<VkWriteDescriptorSet, 3> writes{};
983
984         // UBO
985         writes[0].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
986         writes[0].dstSet = descriptorSets[i];
987         writes[0].dstBinding = 0;
988         writes[0].dstArrayElement = 0;
989         writes[0].descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
990         writes[0].descriptorCount = 1;
991         writes[0].pBufferInfo = &bufferInfo;
992
993         // Binding 1: coin
994         writes[1].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
995         writes[1].dstSet = descriptorSets[i];
996         writes[1].dstBinding = 1;
997         writes[1].dstArrayElement = 0;
998         writes[1].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
999         writes[1].descriptorCount = 1;
1000         writes[1].pImageInfo = &coinInfo;
1001
1002         // Binding 2: tile
1003         writes[2].sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
1004         writes[2].dstSet = descriptorSets[i];
1005         writes[2].dstBinding = 2;
1006         writes[2].dstArrayElement = 0;
1007         writes[2].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
1008         writes[2].descriptorCount = 1;
1009         writes[2].pImageInfo = &tileInfo;
1010
1011         vkUpdateDescriptorSets(device, static_cast<uint32_t>(writes.size()), writes.data(), 0, nullptr);
1012     }
1013 }

```

```

711 void HelloTriangleApplication::createDescriptorSetLayout() {
712     VkDescriptorSetLayoutBinding uboLayoutBinding{};
713     uboLayoutBinding.binding = 0;
714     uboLayoutBinding.descriptorCount = 1;
715     uboLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER; // <- fixed
716     uboLayoutBinding.stageFlags = VK_SHADER_STAGE_VERTEX_BIT;
717     uboLayoutBinding.pImmutableSamplers = nullptr;
718
719     // Binding 1: coin texture
720     VkDescriptorSetLayoutBinding samplerLayoutBinding1{};
721     samplerLayoutBinding1.binding = 1;
722     samplerLayoutBinding1.descriptorCount = 1;
723     samplerLayoutBinding1.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
724     samplerLayoutBinding1.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;
725     samplerLayoutBinding1.pImmutableSamplers = nullptr;
726
727     // Binding 2: tile texture
728     VkDescriptorSetLayoutBinding samplerLayoutBinding2{};
729     samplerLayoutBinding2.binding = 2;
730     samplerLayoutBinding2.descriptorCount = 1;
731     samplerLayoutBinding2.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
732     samplerLayoutBinding2.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;
733     samplerLayoutBinding2.pImmutableSamplers = nullptr;
734
735     std::array<VkDescriptorSetLayoutBinding, 3> bindings = {
736         uboLayoutBinding, samplerLayoutBinding1, samplerLayoutBinding2
737     };
738
739     VkDescriptorSetLayoutCreateInfo layoutInfo{};
740     layoutInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO;
741     layoutInfo.bindingCount = static_cast<uint32_t>(bindings.size());
742     layoutInfo.pBindings = bindings.data();
743 }

```



EXERCISE 8. AN OPEN BOX

Solution & Reflection: This exercise involved creating a single open-topped box with different textures on the inside and outside: rock on the exterior and wood on the interior. This was a multi-stage process that involved fixing several fundamental rendering issues.

First, to create the "open" effect, I modified the draw command in `recordCommandBuffer`. Instead of drawing all 36 indices, I used two separate `vkCmdDrawIndexed` calls: one to draw the first 24 indices (the four side faces) and a second to draw the last 6 indices (the bottom face), specifically skipping the 6 indices corresponding to the top face.

Second, to make the internal faces visible, I had to disable back-face culling. By default, the GPU culls the "inside" of triangles. I disabled this in `createGraphicsPipeline` by setting `rasterizer.cullMode = VK_CULL_MODE_NONE;`

With the inside visible, the core challenge was applying two different textures to the same object. I achieved this in the fragment shader by using the built-in `gl_FrontFacing` variable. The shader logic checks this boolean: if true (the pixel is on an outside face), it samples the `tileTex` (rock); if false (an inside face), it samples the `mainTex` (wood).

This implementation immediately revealed two major bugs:

1. **Winding Order:** The `gl_FrontFacing` logic failed on some faces. This was because the provided `Cube_vertices` data had inconsistent winding order (some faces were Clockwise, some Counter-Clockwise), which "tricked" the shader. I had to manually edit the `Cube_vertices` array to ensure all faces were defined with a consistent Counter-Clockwise (CCW) winding order.
2. **Depth (Z-Fighting):** The box rendered with severe graphical glitches, with the far inside walls drawing on top of the near outside walls. This was because the application lacked a depth buffer.

To solve this, I implemented a full depth-stencil buffer. This required creating a `depthImage`, `depthImageMemory`, and `depthImageView`. I then updated `createGraphicsPipeline` to include a `VkPipelineDepthStencilStateCreateInfo` struct, enabling both `depthTestEnable` and `depthWriteEnable`. Finally, in `recordCommandBuffer`, I attached the `depthImageView` to the `VkRenderingInfo` struct as the `pDepthAttachment`. This also introduced an initialization crash, which I fixed by moving the `createCommandPool()` call to an earlier point in `initVulkan()`, ensuring the command pool existed before `createDepthResources` was called.

This exercise was a practical demonstration of how culling, shader logic (`gl_FrontFacing`), vertex data (winding order), and the depth buffer are all critically interconnected for correct 3D rendering.

Question: Using `gl_FrontFacing` works well here, but it feels like a special case. For a complex model that requires a "two-sided" material (like a piece of cloth or paper), is checking `gl_FrontFacing` in the shader the standard, efficient method? Or do rendering engines prefer to use two separate draw calls (one with `VK_CULL_MODE_FRONT_BIT` and one with `VK_CULL_MODE_BACK_BIT`), or even just duplicate the geometry with flipped normals?

```
rasterizer.lineWidth = 1.0f;|
rasterizer.cullMode = VK_CULL_MODE_NONE;
rasterizer.frontFace = VK_FRONT_FACE_CLOCKWISE;

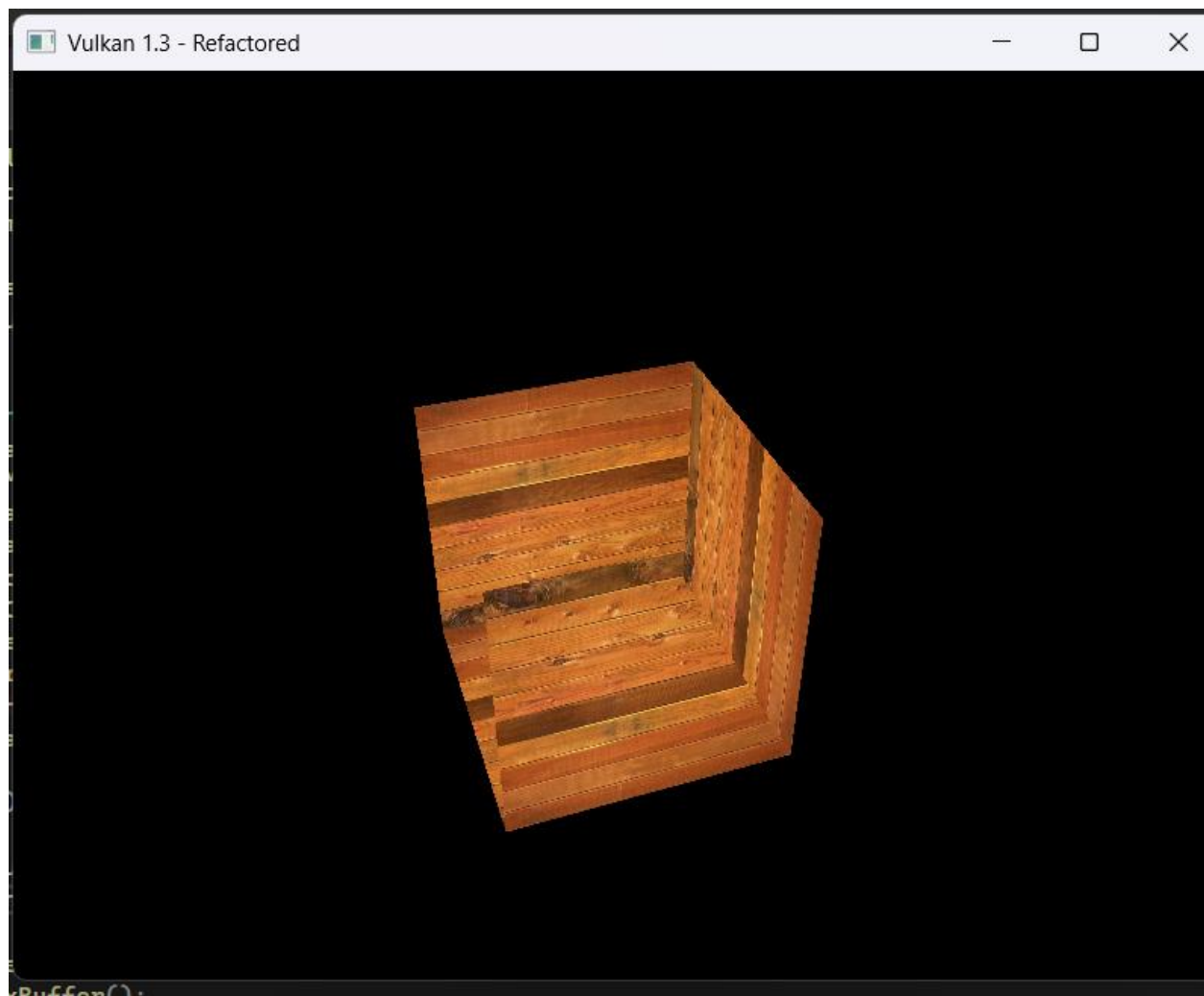
445 loadModel();
446
447 createTextureFromFile("Wood2.jpg");
448 createSecondTextureFromFile("Wall.png");
449
450 createVertexBuffer();
```

```
shader.vert  shader.frag  Lab_Tutorial_Template.cpp
1  #version 450
2
3  layout(location = 0) in vec3 vColor;
4  layout(location = 1) in vec2 vUV;
5
6  layout(location = 0) out vec4 outColor;
7
8  layout(set = 0, binding = 1) uniform sampler2D mainTex;
9  layout(set = 0, binding = 2) uniform sampler2D tileTex;
10
11 void main() {
12     vec3 color = texture(mainTex, vUV).rgb;
13
14     // Repeat the tile texture to add detail
15     vec2 tileUV = vUV * 4.0;
16     vec3 tile = texture(tileTex, tileUV).rgb;
17
18     // Detail multiply (keeps coin as main albedo, modulated by tiles)
19     //vec3 combined = color * (tile * 0.9 + 0.1);
20     outColor = vec4(color, 1.0);
21 }
```

```

82
83 ObjectPushConstants pc = materials[0];
84
85 glm::mat4 model = glm::mat4(1.0f);
86
87 // Rotate it to match the exercise view
88 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
89
90 pc.model = model;
91
92 vkCmdPushConstants(
93     commandBuffer,
94     pipelineLayout,
95     VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT,
96     0,
97     sizeof(ObjectPushConstants),
98     &pc);
99
100 // Draw 30 indices (5 faces) instead of 36 (all 6 faces)
101 // This removes the top face!
102 vkCmdDrawIndexed(commandBuffer, 30, 1, 0, 0, 0);
103

```



Buffer() :

```

// Matches the layout in shader.vert
struct ObjectPushConstants {
    glm::mat4 model;
    alignas(16) glm::vec4 ambientMat;
    alignas(16) glm::vec4 specularMat;
    float shininess;

    alignas(4) int texIndex; // Added for texture array indexing
};

```

```

shader.vert  shader.frag  Lab_Tutorial_Template.cpp*
1      #version 450
2
3      layout(location = 0) in vec3 inPos;
4      layout(location = 1) in vec3 inColor;
5      layout(location = 2) in vec3 inNormal;
6      layout(location = 3) in vec2 inUV;
7
8      layout(location = 0) out vec3 vColor;
9      layout(location = 1) out vec2 vUV;
10     layout(location = 2) out flat int vTexIndex;
11
12     layout(set = 0, binding = 0, std140) uniform SceneUBO {
13         mat4 view;
14         mat4 proj;
15         vec3 lightPos1; float _pad0;
16         vec3 lightPos2; float _pad1;
17         vec3 eyePos;    float _pad2;
18         vec2 texSize;  int  filterMode; float _pad3;
19     } ubo;
20
21     layout(push_constant) uniform ObjectPC {
22         mat4 model;
23         vec4 ambientMat;
24         vec4 specularMat;
25         float shininess;
26         int texIndex; // <-- Add this
27     } pc;
28
29     void main() {
30         vColor = inColor;
31         vUV = inUV;
32         vTexIndex = pc.texIndex;
33         gl_Position = ubo.proj * ubo.view * pc.model * vec4(inPos, 1.0);
34     }

```

```
shader.vert  shader.frag  Lab_Tutorial_Template.cpp*
1      #version 450
2
3      layout(location = 0) in vec3 vColor;
4      layout(location = 1) in vec2 vUV;
5
6      layout(location = 2) in flat int vTexIndex;
7
8      layout(location = 0) out vec4 outColor;
9
10     layout(set = 0, binding = 1) uniform sampler2D mainTex;
11     layout(set = 0, binding = 2) uniform sampler2D tileTex;
12
13     void main() {
14         vec3 color;
15
16         // Use the index to choose the texture
17         if (vTexIndex == 0) {
18             color = texture(mainTex, vUV).rgb;
19         } else {
20             color = texture(tileTex, vUV).rgb;
21         }
22         outColor = vec4(color, 1.0);
23     }
```

```

758
759 void HelloTriangleApplication::createDepthResources()
760 {
761     VkExtent2D extent = swapChainExtent;
762
763     // Create the Image
764     VkImageCreateInfo imageInfo{};
765     imageInfo.sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO;
766     imageInfo.imageType = VK_IMAGE_TYPE_2D;
767     imageInfo.extent.width = extent.width;
768     imageInfo.extent.height = extent.height;
769     imageInfo.extent.depth = 1;
770     imageInfo.mipLevels = 1;
771     imageInfo.arrayLayers = 1;
772     imageInfo.format = depthFormat;
773     imageInfo.tiling = VK_IMAGE_TILING_OPTIMAL;
774     imageInfo.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
775     imageInfo.usage = VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT;
776     imageInfo.samples = VK_SAMPLE_COUNT_1_BIT;
777     imageInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
778
779     if (vkCreateImage(device, &imageInfo, nullptr, &depthImage) != VK_SUCCESS) {
780         throw std::runtime_error("failed to create depth image!");
781     }
782
783     // Allocate Memory
784     VkMemoryRequirements memRequirements;
785     vkGetImageMemoryRequirements(device, depthImage, &memRequirements);
786
787     VkMemoryAllocateInfo allocInfo{};
788     allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
789     allocInfo.allocationSize = memRequirements.size;
790     allocInfo.memoryTypeIndex = findMemoryType(memRequirements.memoryTypeBits, VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT);
791
792     if (vkAllocateMemory(device, &allocInfo, nullptr, &depthImageMemory) != VK_SUCCESS) {
793         throw std::runtime_error("failed to allocate depth image memory!");
794     }
795
796     // Bind Memory
797     vkBindImageMemory(device, depthImage, depthImageMemory, 0);
798
799     // Create the Image View
800     depthImageView = createImageView(depthImage, depthFormat, VK_IMAGE_ASPECT_DEPTH_BIT);
801
802     // Transition Layout
803     transitionImageLayout(depthImage,

```

```

901 multisampling.samples = VK_SAMPLE_COUNT_1_BIT;
902
903     VkPipelineDepthStencilStateCreateInfo depthStencil{};
904     depthStencil.sType = VK_STRUCTURE_TYPE_PIPELINE_DEPTH_STENCIL_STATE_CREATE_INFO;
905     depthStencil.depthTestEnable = VK_TRUE;
906     depthStencil.depthWriteEnable = VK_TRUE;
907     depthStencil.depthCompareOp = VK_COMPARE_OP_LESS; // Nearer objects (less depth) pass
908     depthStencil.depthBoundsTestEnable = VK_FALSE;
909     depthStencil.stencilTestEnable = VK_FALSE;
910

```

```

1328 colorAttachment.storeOp = VK_ATTACHMENT_STORE_OP_STORE;
1329 colorAttachment.clearValue.color = { 0.0f, 0.0f, 0.0f, 1.0f };
1330
1331     VkRenderingAttachmentInfo depthAttachment{};
1332     depthAttachment.sType = VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO;
1333     depthAttachment.imageView = depthImageView;
1334     depthAttachment.imageLayout = VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL;
1335     depthAttachment.loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR; // Clear depth at start of frame
1336     depthAttachment.storeOp = VK_ATTACHMENT_STORE_OP_DONT_CARE; // We don't need it after
1337     depthAttachment.clearValue.depthStencil = { 1.0f, 0 };
1338
1339     VkRenderingInfo renderingInfo{};

```

```

1405
1406 ObjectPushConstants pc = materials[0]; // Use the same material properties
1407
1408 // --- Draw ONE Box (Rock Outside, Wood Inside) ---
1409 glm::mat4 model = glm::mat4(1.0f);
1410 // model = glm::translate(model, glm::vec3(-1.0f, 0.0f, 0.0f)); // REMOVE THIS
1411 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
1412
1413 pc.model = model;
1414 pc.texIndex = 1; // <-- Tell the box to use texture 1 (rock)
1415
1416 vkCmdPushConstants(
1417     commandBuffer,
1418     pipelineLayout,
1419     VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT,
1420     0,
1421     sizeof(ObjectPushConstants),
1422     &pc);
1423
1424 vkCmdDrawIndexed(commandBuffer, 24, 1, 0, 0, 0);
1425
1426 // Draw the 6th face (Bottom) - 6 indices, starting from index 30
1427 vkCmdDrawIndexed(commandBuffer, 6, 1, 30, 0, 0);
1428

```